1. PID CONTROLLER

- PROJECT 1: PID CONTROLLER FOR TEMPERATURE

This project involves the development of a closed-loop temperature control system for maintaining a metallic (Aluminum) heating block at a specified temperature. A **K-type thermocouple** is used to measure the actual temperature of the block, while a thermocouple interface module such as the **MAX6675** is used for signal acquisition. An Arduino microcontroller processes the measured temperature and implements a PID control algorithm to regulate the system. Power delivery to the heating element is controlled through a switching device such as a **MOSFET** for DC loads or a **TRIAC** for high-voltage AC applications. The controller continuously compares the measured temperature with the desired setpoint and adjusts the power supplied to the heating element accordingly, forming a feedback-based closed-loop control system.

HOW PID CONTROL WORKS: A PID temperature controller operates by continuously monitoring and regulating the temperature of a system to achieve a desired target value. In this project, the controlled variable is the temperature of the heating block. To maintain accurate temperature regulation, the system requires a feedback mechanism, which is provided by a K-type thermocouple. The thermocouple measures the actual temperature of the system in real time and sends this information to the controller. The controller also requires a setpoint, which represents the desired temperature to be achieved and maintained. The PID controller continuously compares the measured temperature from the thermocouple with the setpoint value. The difference between these two values is known as the error. Based on this error, the controller calculates the appropriate control action using three components: proportional (P), integral (I), and derivative (D). The proportional term reacts to the present error, the integral term considers the accumulation of past errors, and the derivative term predicts future error behavior based on the rate of change. By combining these three actions, the PID controller adjusts the power delivered to the heating element, allowing the system to reach the target temperature efficiently while minimizing overshoot and oscillations.

If only proportional control is used, the system operates as a **P-controller**. In temperature regulation systems, proportional control alone often causes continuous oscillations around the desired temperature, making the system unstable or unable to maintain a precise steady-state value. To improve stability and response, **derivative control (D)** is introduced. The derivative component responds to the rate of temperature change, allowing the controller to react quickly to sudden

disturbances. For example, if external cooling occurs, such as airflow over the aluminum block, the derivative action rapidly increases the heating power to counteract the temperature drop and maintain the setpoint. **The integral component (I)** is added to eliminate steady-state error. This term continuously accumulates the error over time, increasing when the measured temperature remains below the setpoint and decreasing when the error becomes negative. By integrating the error across successive control loops, the controller compensates for long-term deviations and improves accuracy. The combination of the proportional, integral, and derivative actions forms a PID controller. The overall performance of the system depends on selecting suitable values for the PID constants, commonly referred to as **tuning parameters**. Proper tuning ensures stable operation, fast response, reduced overshoot, and accurate temperature control.

K-TYPE THERMOCOUPLE TEMPERATURE MEASUREMENT: The first stage in developing the temperature control system is obtaining accurate real-time temperature measurements. In this project, a K-type thermocouple is used as the temperature sensing element together with the MAX6675 thermocouple interface module. The thermocouple generates a very small voltage proportional to temperature, while the MAX6675 module amplifies, compensates, and converts this signal into a digital value that can be read by the microcontroller. The MAX6675 communicates with the Arduino through **the SPI (Serial Peripheral Interface) protocol**. Therefore, the module pins must be connected to the corresponding SPI pins of the Arduino, together with the power supply and ground connections. During hardware setup, the thermocouple is connected directly to the MAX6675 module. Correct polarity must be observed by connecting the positive and negative thermocouple terminals to their corresponding module inputs. When heat is applied to the thermocouple, for example using a lighter or another heat source, the displayed temperature increases accordingly, confirming successful real-time temperature acquisition from the sensor system.

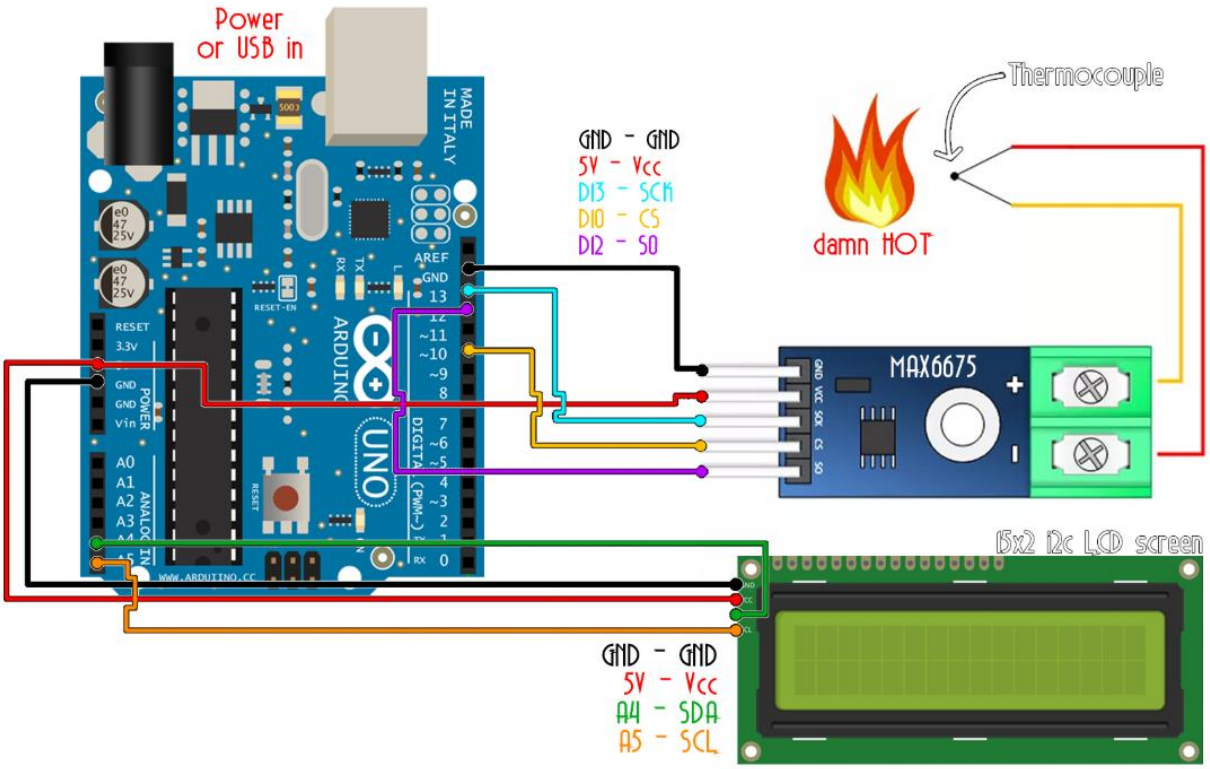

Figure 1: K Thermocouple temp. read

```
/*    Max6675 Module  ==>   Arduino
 *    CS              ==>    D10
 *    SO              ==>    D12
 *    SCK              ==>    D13
 *    Vcc             ==>    Vcc (5v)
 *    Gnd             ==>    Gnd     */
//LCD config
#include <Wire.h>
#include <LiquidCrystal_I2C.h>    //If you don't have the LiquidCrystal_I2C library, download it and install it
LiquidCrystal_I2C lcd(0x3f,20,4);  //sometimes the adress is not 0x3f. Change to 0x27 if it dosn't work.
/*    i2c LCD Module  ==>   Arduino
 *    SCL             ==>    A5
 *    SDA             ==>    A4
```

```
 *    Vcc            ==>     Vcc (5v)
 *    Gnd            ==>     Gnd      */

#include <SPI.h>
#define MAX6675_CS   10
#define MAX6675_SO   12
#define MAX6675_SCK  13
void setup() {
  lcd.init();
  lcd.backlight();
}
void loop() {
  float temperature_read = readThermocouple();
  lcd.setCursor(0,0);
  lcd.print("TEMPERATURE");
  lcd.setCursor(7,1);
  lcd.print(temperature_read,1);
  delay(300);
}
double readThermocouple() {
  uint16_t v;
  pinMode(MAX6675_CS, OUTPUT);
  pinMode(MAX6675_SO, INPUT);
  pinMode(MAX6675_SCK, OUTPUT);
  digitalWrite(MAX6675_CS, LOW);
  delay(1);
  // Read in 16 bits,
  //  15    = 0 always
  //  14..2 = 0.25 degree counts MSB First
  //  2     = 1 if thermocouple is open circuit
  //  1..0  = uninteresting status
```

```
  v = shiftIn(MAX6675_SO, MAX6675_SCK, MSBFIRST);
  v <<= 8;
  v |= shiftIn(MAX6675_SO, MAX6675_SCK, MSBFIRST);
  digitalWrite(MAX6675_CS, HIGH);
  if (v & 0x4)
  {
    // Bit 2 indicates if the thermocouple is disconnected
    return NAN;
  }
  // The lower three bits (0,1,2) are discarded status bits
  v >>= 3;
  // The remaining bits are the number of 0.25 degree (C) counts
  return v*0.25;
}
```

PID TEMPERATURE CONTROL IMPLEMENTATION: After successfully obtaining real-time temperature measurements from the thermocouple, the next stage is implementing closed-loop PID temperature control. In this stage, the heating element power is regulated using a MOSFET switching circuit controlled by an Arduino microcontroller. The Arduino continuously reads the actual temperature from the MAX6675 thermocouple module, computes the PID control output, and generates a PWM (Pulse Width Modulation) signal to regulate the heating power. The PID algorithm operates by first calculating the error between the desired temperature setpoint and the measured temperature:

$$e(t)=T_{set} - T_{measured} \quad \text{.......} \quad (1)$$

The proportional term responds directly to the current error, the integral term accumulates past errors over time, and the derivative term predicts future system behavior by considering the rate of temperature change. The total PID output is obtained by summing these three contributions:

$$PID=K_p e(t) + K_i \int e(t)dt + K_d \frac{de(t)}{dt} \quad \text{.......} \quad (2)$$

The resulting PID value is converted into a PWM signal generated on Arduino digital pin D3. This PWM signal drives the gate of an N-channel MOSFET through a transistor driver stage, thereby controlling the amount of power delivered to the heating element. By continuously adjusting the

PWM duty cycle, the system maintains the aluminum block temperature close to the specified setpoint. An LCD display is used to monitor both the desired temperature and the measured real-time temperature. During operation, the controller initially applies significant power to raise the temperature rapidly toward the setpoint. As the measured temperature approaches the desired value, the PID controller reduces the PWM duty cycle to avoid excessive overshoot and stabilize the system. The effectiveness of the PID controller depends strongly on the tuning of the proportional, integral, and derivative constants. Improper values may lead to oscillations, slow response, or unstable behavior. A common tuning approach is to begin with only proportional control by setting the integral and derivative gains to zero, then gradually increasing the integral and derivative terms until stable and accurate temperature regulation is achieved.

Experimental observation of the PWM signal using an oscilloscope shows that the duty cycle changes dynamically according to system conditions. When the heating block is externally cooled, such as by airflow, the controller automatically adjusts the PWM signal to compensate for heat loss and maintain the target temperature. This demonstrates the effectiveness of the closed-loop PID temperature control system.

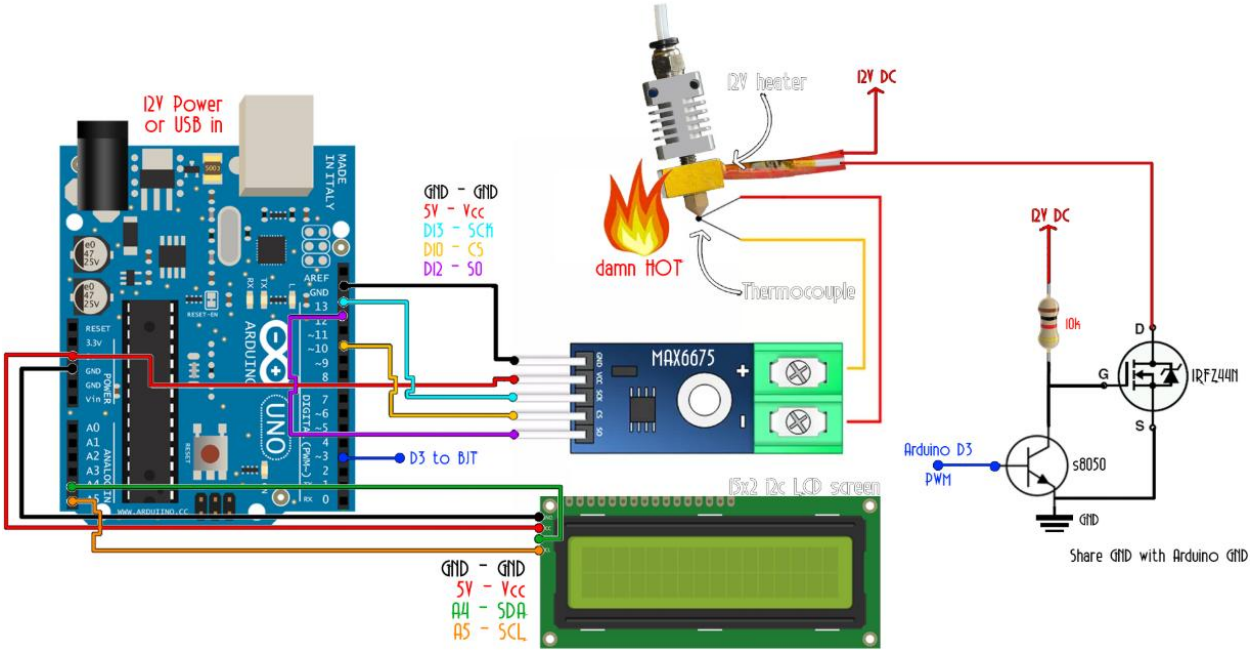

Figure 2: PID temperature control

```
/*    Max6675 Module  ==>   Arduino
 *    CS             ==>    D10
 *    SO             ==>    D12
 *    SCK             ==>    D13
 *    Vcc            ==>    Vcc (5v)
 *    Gnd            ==>    Gnd      */

//LCD config (i2c LCD screen, you need to install the LiquidCrystal_I2C if you don't have it )
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x3f,20,4);  //sometimes the adress is not 0x3f. Change to 0x27 if it dosn't work.

/*    i2c LCD Module  ==>   Arduino
 *    SCL            ==>    A5
 *    SDA             ==>    A4
 *    Vcc            ==>    Vcc (5v)
 *    Gnd            ==>    Gnd      */

#include <SPI.h>
//We define the SPI pìns
#define MAX6675_CS   10
#define MAX6675_SO   12
#define MAX6675_SCK  13
//Pins
int PWM_pin = 3;
//Variables
float temperature_read = 0.0;
float set_temperature = 100;
float PID_error = 0;
float previous_error = 0;
```

```
float elapsedTime, Time, timePrev;
int PID_value = 0;
//PID constants
int kp = 9.1;   int ki = 0.3;   int kd = 1.8;
int PID_p = 0;    int PID_i = 0;    int PID_d = 0;
void setup() {
  pinMode(PWM_pin,OUTPUT);
  TCCR2B = TCCR2B & B11111000 | 0x03;    // pin 3 and 11 PWM frequency of 980.39 Hz
  Time = millis();
  lcd.init();
  lcd.backlight();
}
void loop() {
 // First we read the real value of temperature
  temperature_read = readThermocouple();
  //Next we calculate the error between the setpoint and the real value
  PID_error = set_temperature - temperature_read;
  //Calculate the P value
  PID_p = kp * PID_error;
  //Calculate the I value in a range on +-3
  if(-3 < PID_error <3)
  {
    PID_i = PID_i + (ki * PID_error);
  }

  //For derivative we need real time to calculate speed change rate
  timePrev = Time;                        // the previous time is stored before the actual time read
  Time = millis();                        // actual time read
  elapsedTime = (Time - timePrev) / 1000;
  //Now we can calculate the D calue
  PID_d = kd*((PID_error - previous_error)/elapsedTime);
```

```
  //Final total PID value is the sum of P + I + D
  PID_value = PID_p + PID_i + PID_d;

  //We define PWM range between 0 and 255
  if(PID_value < 0)
  {    PID_value = 0;    }
  if(PID_value > 255)
  {    PID_value = 255;  }
  //Now we can write the PWM signal to the mosfet on digital pin D3
  analogWrite(PWM_pin,255-PID_value);
  previous_error = PID_error;     //Remember to store the previous error for next loop.
  delay(300);
  //lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("PID TEMP control");
  lcd.setCursor(0,1);
  lcd.print("S:");
  lcd.setCursor(2,1);
  lcd.print(set_temperature,1);
  lcd.setCursor(9,1);
  lcd.print("R:");
  lcd.setCursor(11,1);
  lcd.print(temperature_read,1);
}
double readThermocouple() {

  uint16_t v;
  pinMode(MAX6675_CS, OUTPUT);
  pinMode(MAX6675_SO, INPUT);
  pinMode(MAX6675_SCK, OUTPUT);
```

```
 digitalWrite(MAX6675_CS, LOW);
 delay(1);

 // Read in 16 bits,
 //  15    = 0 always
 //  14..2 = 0.25 degree counts MSB First
 //  2     = 1 if thermocouple is open circuit
 //  1..0  = uninteresting status

 v = shiftIn(MAX6675_SO, MAX6675_SCK, MSBFIRST);
 v <<= 8;
 v |= shiftIn(MAX6675_SO, MAX6675_SCK, MSBFIRST);

 digitalWrite(MAX6675_CS, HIGH);
 if (v & 0x4)
 {
   // Bit 2 indicates if the thermocouple is disconnected
   return NAN;
 }
 // The lower three bits (0,1,2) are discarded status bits
 v >>= 3;

 // The remaining bits are the number of 0.25 degree (C) counts
 return v*0.25;
}
```

PID WITH THE ROTARY CONTROL: The system integrates a user interface and control stage to allow real-time monitoring and adjustment of a temperature-based PID controller. A rotary encoder with an integrated push button is used to emulate the functionality of a commercial PID controller, enabling navigation through parameters and modification of values. The system includes an LCD display for real-time temperature visualization, a thermocouple with a MAX6675 module for accurate temperature feedback, and a power stage consisting of a BJT-driven MOSFET that regulates a DC heating element. The thermocouple must be properly attached to the heating element to ensure accurate thermal measurement and minimize delay between the actual and measured temperatures. Through the rotary encoder, the user can set the desired temperature and adjust PID constants (Kp, Ki, and Kd) by cycling through a menu system. Once configured, the controller continuously compares the measured temperature with the setpoint and computes a PID output that drives a PWM signal to the MOSFET, regulating the heater's power. As the system approaches the setpoint, small PWM variations occur to maintain thermal stability due to feedback correction and the inherent thermal inertia of the heating element. The design is intended for DC heating applications only and is not suitable for direct AC mains control without additional circuitry such as a TRIAC-based switching stage

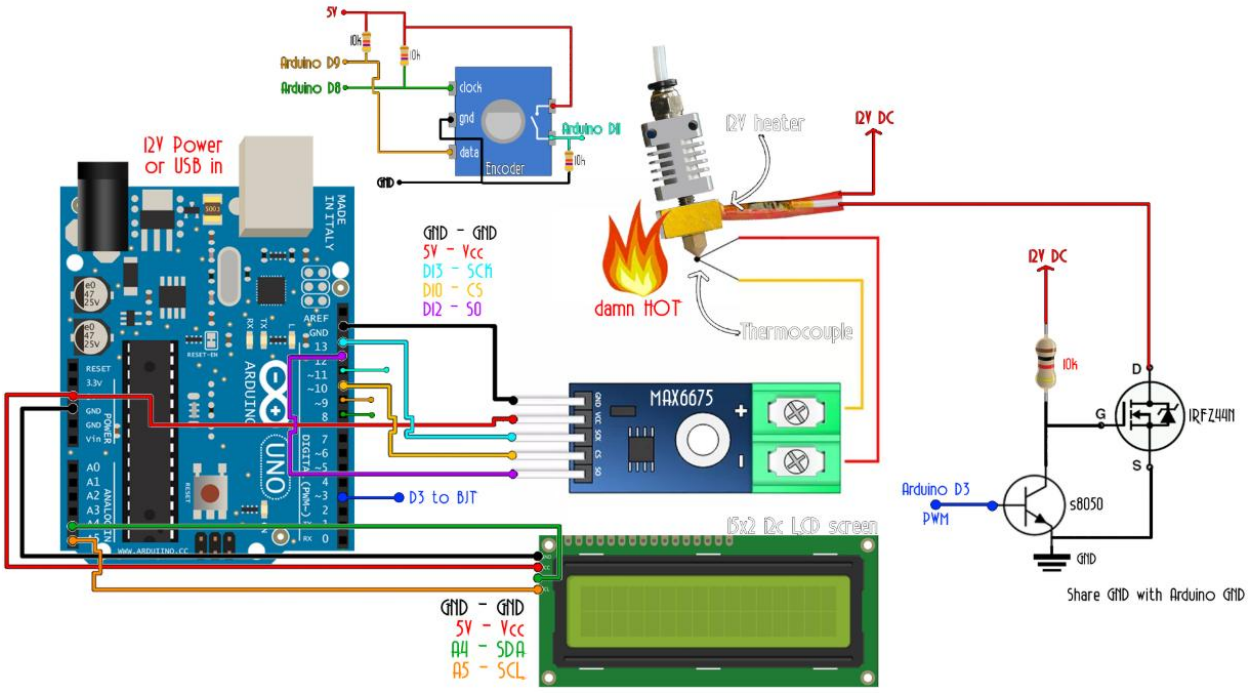

Figure 3: PID with rotary encoder

```
/*    Max6675 Module  ==>   Arduino
 *    CS             ==>    D10
 *    SO             ==>    D12
 *    SCK             ==>    D13
 *    Vcc            ==>    Vcc (5v)
 *    Gnd            ==>    Gnd      */
#include <SPI.h>

//LCD config
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x3f,20,4);  //sometimes the adress is not 0x3f. Change to 0x27 if it dosn't work.

/*    i2c LCD Module  ==>   Arduino
 *    SCL            ==>    A5
 *    SDA             ==>    A4
 *    Vcc            ==>    Vcc (5v)
 *    Gnd            ==>    Gnd      */

//I/O
int PWM_pin = 3;  //Pin for PWM signal to the MOSFET driver (the BJT npn with pullup)
int clk = 8;      //Pin 1 from rotary encoder
int data = 9;     //Pin 2 from rotary encoder

//Variables
float set_temperature = 0;            //Default temperature setpoint. Leave it 0 and control it with rotary encoder

float temperature_read = 0.0;
float PID_error = 0;
```

```
float previous_error = 0;
float elapsedTime, Time, timePrev;
float PID_value = 0;
int button_pressed = 0;
int menu_activated=0;
float last_set_temperature = 0;

//Vraiables for rotary encoder state detection
int clk_State;
int Last_State;
bool dt_State;

//PID constants
int kp = 90;   int ki = 30;   int kd = 80;

int PID_p = 0;    int PID_i = 0;    int PID_d = 0;
float last_kp = 0;
float last_ki = 0;
float last_kd = 0;

int PID_values_fixed =0;

//Pins for the SPI with MAX6675
#define MAX6675_CS   10
#define MAX6675_SO   12
#define MAX6675_SCK  13

void setup() {
  pinMode(PWM_pin,OUTPUT);
  TCCR2B = TCCR2B & B11111000 | 0x03;    // pin 3 and 11 PWM frequency of 928.5 Hz
  Time = millis();
```

```
  Last_State = (PINB & B00000001);      //Detect first state of the encoder

  PCICR |= (1 << PCIE0);    //enable PCMSK0 scan
  PCMSK0 |= (1 << PCINT0);  //Set pin D8 trigger an interrupt on state change.
  PCMSK0 |= (1 << PCINT1);  //Set pin D9 trigger an interrupt on state change.
  PCMSK0 |= (1 << PCINT3);  //Set pin D11 trigger an interrupt on state change.

  pinMode(11,INPUT);
  pinMode(9,INPUT);
  pinMode(8,INPUT);

  lcd.init();
  lcd.backlight();
}

void loop() {


if(menu_activated==0)
{
  // First we read the real value of temperature
  temperature_read = readThermocouple();
  //Next we calculate the error between the setpoint and the real value
  PID_error = set_temperature - temperature_read + 3;
  //Calculate the P value
  PID_p = 0.01*kp * PID_error;
  //Calculate the I value in a range on +-3
  PID_i = 0.01*PID_i + (ki * PID_error);

  //For derivative we need real time to calculate speed change rate
```

```
  timePrev = Time;                        // the previous time is stored before the actual time read
  Time = millis();                        // actual time read
  elapsedTime = (Time - timePrev) / 1000;
  //Now we can calculate the D calue
  PID_d = 0.01*kd*((PID_error - previous_error)/elapsedTime);
  //Final total PID value is the sum of P + I + D
  PID_value = PID_p + PID_i + PID_d;

  //We define PWM range between 0 and 255
  if(PID_value < 0)
  {    PID_value = 0;    }
  if(PID_value > 255)
  {    PID_value = 255;  }
  //Now we can write the PWM signal to the mosfet on digital pin D3
  //Since we activate the MOSFET with a 0 to the base of the BJT, we write 255-PID value (inverted)
  analogWrite(PWM_pin,255-PID_value);
  previous_error = PID_error;     //Remember to store the previous error for next loop.

  delay(250); //Refresh rate + delay of LCD print
  //lcd.clear();

  lcd.setCursor(0,0);
  lcd.print("PID TEMP control");
  lcd.setCursor(0,1);
  lcd.print("S:");
  lcd.setCursor(2,1);
  lcd.print(set_temperature,1);
  lcd.setCursor(9,1);
  lcd.print("R:");
  lcd.setCursor(11,1);
```

```
  lcd.print(temperature_read,1);
}//end of menu 0 (PID control)

//First page of menu (temp setpoint)
if(menu_activated == 1)
{
   analogWrite(PWM_pin,255);
  if(set_temperature != last_set_temperature)
  {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("Set  temperature");
  lcd.setCursor(0,1);
  lcd.print(set_temperature);
  }
  last_set_temperature = set_temperature;


}//end of menu 1

//Second page of menu (P set)
if(menu_activated == 2)
{

  if(kp != last_kp)
  {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("Set   P  value  ");
  lcd.setCursor(0,1);
  lcd.print(kp);
```

```
  }
  last_kp = kp;

}//end of menu 2

//Third page of menu (I set)
if(menu_activated == 3)
{

  if(ki != last_ki)
  {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("Set   I  value  ");
  lcd.setCursor(0,1);
  lcd.print(ki);
  }
  last_ki = ki;

}//end of menu 3
//Forth page of menu (D set)
if(menu_activated == 4)
{

  if(kd != last_kd)
  {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("Set   D  value  ");
  lcd.setCursor(0,1);
  lcd.print(kd);
```

```
  }
  last_kd = kd;
}//end of menu 4

}//Loop end

//The function that reads the SPI data from MAX6675
double readThermocouple() {

  uint16_t v;
  pinMode(MAX6675_CS, OUTPUT);
  pinMode(MAX6675_SO, INPUT);
  pinMode(MAX6675_SCK, OUTPUT);

  digitalWrite(MAX6675_CS, LOW);
  delay(1);

  // Read in 16 bits,
  //  15    = 0 always
  //  14..2 = 0.25 degree counts MSB First
  //  2     = 1 if thermocouple is open circuit
  //  1..0  = uninteresting status

  v = shiftIn(MAX6675_SO, MAX6675_SCK, MSBFIRST);
  v <<= 8;
  v |= shiftIn(MAX6675_SO, MAX6675_SCK, MSBFIRST);

  digitalWrite(MAX6675_CS, HIGH);
  if (v & 0x4)
  {
    // Bit 2 indicates if the thermocouple is disconnected
```

```
  return NAN;
 }

 // The lower three bits (0,1,2) are discarded status bits
 v >>= 3;

 // The remaining bits are the number of 0.25 degree (C) counts
 return v*0.25;
}
```

```
//The interruption vector for push button and rotary encoder
ISR(PCINT0_vect){
if(menu_activated==1)
  {
 clk_State =   (PINB & B00000001); //pin 8 state? It is HIGH?
 dt_State  =   (PINB & B00000010);
 if (clk_State != Last_State){
   // If the data state is different to the clock state, that means the encoder is rotating clockwise
   if (dt_State != clk_State) {
     set_temperature = set_temperature+0.5 ;
   }
   else {
     set_temperature = set_temperature-0.5;
```

```
    }
  }
  Last_State = clk_State; // Updates the previous state of the clock with the current state
}

if(menu_activated==2)
   {
  clk_State =   (PINB & B00000001); //pin 8 state?
  dt_State  =   (PINB & B00000010);
  if (clk_State != Last_State){
     // If the data state is different to the clock state, that means the encoder is rotating clockwise
     if (dt_State != clk_State) {
       kp = kp+1 ;
     }
     else {
       kp = kp-1;
     }
  }
  Last_State = clk_State; // Updates the previous state of the clock with the current state
}

if(menu_activated==3)
   {
  clk_State =   (PINB & B00000001); //pin 8 state?
  dt_State  =   (PINB & B00000010);
  if (clk_State != Last_State){
     // If the data state is different to the clock state, that means the encoder is rotating clockwise
     if (dt_State != clk_State) {
       ki = ki+1 ;
     }
     else {
```

```
      ki = ki-1;
    }
  }
  Last_State = clk_State; // Updates the previous state of the clock with the current state
}

 if(menu_activated==4)
   {
  clk_State =   (PINB & B00000001); //pin 8 state?
  dt_State  =   (PINB & B00000010);
  if (clk_State != Last_State){
    // If the data state is different to the clock state, that means the encoder is rotating clockwise
    if (dt_State != clk_State) {
      kd = kd+1 ;
    }
    else {
      kd = kd-1;
    }
  }
  Last_State = clk_State; // Updates the previous state of the clock with the current state
}


  //Push button was pressed!
  if (PINB & B00001000) //Pin D11 is HIGH?
  {
    button_pressed = 1;
  }
  //We navigate through the 4 menus with each button pressed
  else if(button_pressed == 1)
  {
```

```
if(menu_activated==4)
{
 menu_activated = 0;
 PID_values_fixed=1;
 button_pressed=0;
 delay(1000);
}

if(menu_activated==3)
{
 menu_activated = menu_activated + 1;
 button_pressed=0;
 kd = kd + 1;
 delay(1000);
}

if(menu_activated==2)
{
 menu_activated = menu_activated + 1;
 button_pressed=0;
 ki = ki + 1;
 delay(1000);
}

if(menu_activated==1)
{
 menu_activated = menu_activated + 1;
 button_pressed=0;
 kp = kp + 1;
 delay(1000);
```

```
  }

  if(menu_activated==0 && PID_values_fixed != 1)
  {
   menu_activated = menu_activated + 1;
   button_pressed=0;
   set_temperature = set_temperature+1;
   delay(1000);
  }
  PID_values_fixed = 0;

 }
}
```